Switchboard — Cross-Application Compatibility Report

Assessment of native support across AI coding agents, with verification status per claim.

Field	Value
Report date	2026-08-25
Subject	Switchboard — ambient shared working memory for concurrent agent sessions
Question assessed	*Will it work across all applications natively?*
Verdict	**Yes at Tier 1 or above on every mainstream agent. No per-agent adapters required.**
Companion doc	`switchboard-prd.md`

---

1. Verdict

Yes — with one honest asymmetry.

Native cross-application support is achievable because two universal standards already carry the entire load:

- **MCP** — spoken by every mainstream coding agent. Provides discovery, identity, liveness, and the full read/write interface.
- **AGENTS.md** — a Linux Foundation standard read natively by 30+ agents. Provides the cold-start instruction.

Neither requires a per-agent adapter. An agent released next year that speaks MCP is supported on day one, with no code change from us.

The asymmetry: *automatic* file-claim observation requires lifecycle hooks. Four agents have them. Everyone else gets *voluntary* claims, where the model calls `sb_claim()` because AGENTS.md told it to. This is a real reliability difference and is surfaced in the UI rather than hidden.

Everything else — discovery, roster, digests, search, notes, handoff — is identical on every agent.

---

2. Confidence legend

Each claim below carries a verification status. This is the part of a compatibility report that actually matters.

Mark	Meaning
✅ **V**	**Verified** — confirmed against official vendor documentation
🟡 **S**	**Secondary** — confirmed via multiple independent secondary sources, not vendor docs
⚪ **U**	**Unverified** — inferred or single-source; must be tested before shipping
❌	Confirmed absent

3. The matrix

Agent	MCP	Hooks	AGENTS.md	Tier	Confidence
Claude Code	✅	✅	via `@import`	**T3**	✅ V
Cursor	✅	✅ v1.7+	✅	**T3**	✅ V
Gemini CLI	✅	✅ v0.26+	✅	**T3**	✅ V
OpenCode	✅	✅ 25+ events	✅	**T3**	🟡 S
Windsurf	✅	❌	✅	T1	🟡 S
Codex CLI	✅	❌ *(only major agent without)*	✅	T1	🟡 S
GitHub Copilot / CLI	✅	❌	✅	T1	🟡 S
Cline	✅	❌	✅	T1	🟡 S
Roo Code / Kilo Code	✅	❌	✅	T1	🟡 S
Continue.dev	✅	❌	✅	T1	🟡 S
Zed	✅ `context_servers`	❌	✅	T1	🟡 S
Goose	✅	❌	✅	T1	🟡 S
Qwen Code	✅	❌	✅	T1	🟡 S
Amazon Q / Kiro CLI	✅	❌	⚪	T1	🟡 S
Aider	⚪	❌	✅	T0–T1	⚪ U
Jules / Devin / Factory	⚪	❌	✅	T0	⚪ U
Any future MCP agent	✅	—	—	**T1**	—

The last row is the design goal. Support is a property of the standards, not of a maintained list.

Coverage summary

	Agents	Share of matrix
T3 — fully automatic	4	24%
T1 — full feature set, voluntary claims	10	59%
T0 — instruction-only fallback	3	17%
Excluded entirely	**0**	**0%**

---

4. What each tier actually delivers

Capability	T0	T1	T3
Session appears in roster	❌	✅	✅
Reads other sessions' state	✅ via file	✅ via MCP	✅ via MCP
Cold-start priming	manual read	on request	**automatic**
Search prior findings	❌	✅	✅
Post findings (`sb_note`)	❌	✅	✅
File claims	❌	voluntary	**observed**
Collision warning	❌	on request	**automatic, pre-edit**
Digest on session end	❌	partial	✅
Pre-compaction rescue	❌	❌	✅ Claude Code, Gemini CLI

T1 is a complete product, not a degraded one. Every feature is present; the difference is whether the agent is *told* or has to *ask*.

---

5. Why no adapters are needed

5.1 MCP gives discovery for free

Under stdio transport, an MCP client **spawns one server process per session** and terminates it at session end. The `initialize` handshake carries `clientInfo: { name, version }`.

Requirement	Derived from	Per-agent code
Session discovery	server process starts	none
Agent identity + version	`initialize` → `clientInfo`	none
Workspace scope	server process `cwd`	none
Liveness	process is alive	none
Death detection	process exits	none
Read/write interface	MCP tools	none

This is the entire basis for the "no adapters" claim. It is a property of the MCP stdio lifecycle, not a trick.

5.2 AGENTS.md gives cold-start instruction for free

- Released by OpenAI, August 2025
- Transferred to the Linux Foundation's **Agentic AI Foundation**, 9 December 2025
- By May 2026: **170+ member organisations, 60,000+ repositories, 30+ agents reading it natively**

`init` appends one static stanza, written once, never modified:

```
## Concurrent sessions
Other AI sessions may be working in this repo right now.
Before starting a task, call `sb_peers` (MCP) — or read
`.switchboard/ROSTER.md` — to see what they're doing and which
files they've claimed. Post findings with `sb_note`.
```

All churning state lives in git-excluded `.switchboard/ROSTER.md`, so the repository sees **zero diff noise**.

> **Claude Code exception:** as of August 2026 Claude Code loads `CLAUDE.md`, not `AGENTS.md`. Bridge via a first-line `@AGENTS.md` import or `ln -s AGENTS.md CLAUDE.md` — the approach Anthropic's own documentation recommends.

5.3 Hook lifecycles converged

The four hook-capable agents adopted near-identical event models. One implementation with a ~90-line name mapping covers all four:

Job	Claude Code	Cursor	Gemini CLI	OpenCode
Session start	`SessionStart`	`sessionStart`	`SessionStart`	session event
Before tool	`PreToolUse`	`preToolUse`	`BeforeTool`	tool.before
After edit	`PostToolUse`	`afterFileEdit`	`AfterTool`	tool.after
Turn end	`Stop`	`stop`	`AfterAgent`	agent end
Before compaction	`PreCompact`	❌	`PreCompress`	varies
Session end	`SessionEnd`	`sessionEnd`	`SessionEnd`	session end
Prompt submit	`UserPromptSubmit`	`beforeSubmitPrompt`	`BeforeAgent`	prompt event

Gemini CLI hooks are **on by default** from v0.26.0+ and explicitly support context injection via `BeforeAgent` / `BeforeModel`. OpenCode exposes **25+ lifecycle events** with tool-execution interception through TypeScript plugins.

---

6. The one genuine incompatibility

File-claim attribution cannot be made universal.

Approach	Detects change	Attributes to session	Verdict
Lifecycle hooks	✅	✅	Works — T3 only
Voluntary `sb_claim()`	✅	✅	Works — depends on model compliance
Filesystem watching	✅	❌	Cannot attribute
`git status` polling	✅	❌	Cannot attribute
Editor integration	✅	❌	Editor ≠ agent session

Attribution requires interception, and only hooks intercept. There is no workaround; presenting one would be dishonest.

Mitigation: the roster labels the provenance of every claim, so a reader knows exactly how much to trust it.

```
• data-migration  [codex]  active 22m — backfilling tenant_id
  claimed: scripts/backfill.ts  (self-declared)

• auth-fix  [claude-code]  active 40m — debugging login 500s
  editing: middleware/session.ts, lib/jwt.ts  (observed)
```

---

7. The real integration cost

MCP is universal. MCP *configuration* is not. Every agent uses a different file and format:

Agent	MCP config path	Format
Claude Code	`.mcp.json` / `.claude/settings.json`	JSON
Cursor	`.cursor/mcp.json`	JSON
Windsurf	`~/.codeium/windsurf/mcp_config.json`	JSON
Codex CLI	`~/.codex/config.toml`	**TOML**
Gemini CLI	`.gemini/settings.json`	JSON
VS Code / Copilot	`.vscode/mcp.json`	JSON
Zed	`settings.json` → `context_servers`	JSON
Cline / Roo	extension storage	JSON

This is the total portability tax: **~200 lines of dumb writer functions**, isolated in `installers/`. No cleverness is available or needed.

Consequences for `init`, all mandatory:

- **Idempotent** — re-running changes nothing
- **Diff-printing** — reports exactly which files it touched
- `--dry-run` — inspect before writing
- **Per-agent failure** — one unwritable config never aborts the others
- **Fully reversible** — `uninstall` reverses every write across every agent

---

8. Platform compatibility

OS	T1 (MCP)	T3 (hooks)	`sb_handoff`
macOS	✅	✅	✅
Linux	✅	✅	✅
WSL2	✅	✅	✅
Native Windows	✅	✅	⚠️ deferred

Only `sb_handoff` is OS-gated — it rides Claude Code's native messaging, which uses named pipes and a separate home directory on Windows. Everything else is cross-platform because MCP and file I/O are.

Container boundary: a session inside a container and one on the host cannot see each other — separate filesystems, therefore separate stores. Two sessions inside the same container work normally. Same constraint applies to Claude Code's own cross-session messaging.

---

9. Risks to this assessment

Risk	Severity	Mitigation
MCP config formats change per vendor	**Medium**	Isolated writer functions; `doctor` re-validates; failure is loud and per-agent
T1 voluntary claims ignored by the model	**Medium**	Provenance labelling; explicit AGENTS.md instruction; measured at M2
⚪ Unverified rows (Aider, Jules, Devin, Factory) are wrong	Low	Test before advertising. **Do not claim support for an untested agent.**
Hook APIs drift across four vendors instead of one	Medium	Adapter layer per agent (~25 LOC each); T3 degrades to T1, never to broken
AGENTS.md loses standard status	Low	170+ orgs, Linux Foundation governance. MCP path is independent of it.

---

10. Recommendations

1. **Build T1 first, on Claude Code only.** The MCP server is the substrate; validate it on one agent before touching config writers for seven.
2. **Add T3 for Claude Code, then Cursor.** Same hook shape, second implementation is nearly free.
3. **Verify Gemini CLI and OpenCode hooks by hand** before advertising T3 for them — both are 🟡, not ✅.
4. **Do not claim support for ⚪ rows** until tested. A compatibility table is a promise.
5. **Ship `doctor` in V0.** Users on 7 different agents will hit 7 different config problems, and hook-based tools fail silently by design.
6. **Label claim provenance in the roster from day one.** Retrofitting trust after users discover the difference themselves is much harder than disclosing it upfront.

---

11. Sources

- Hooks reference — Claude Code Docs · ✅ V
- Cross-session messaging — Claude Code Docs · ✅ V
- Agent teams — Claude Code Docs · ✅ V

- Hooks — Cursor Docs · ✅ V
- Gemini CLI hooks — official docs · ✅ V
- Tailor Gemini CLI to your workflow with hooks — Google Developers Blog · ✅ V
- Cursor 1.7 adds hooks for agent lifecycle control — InfoQ · 🟡 S
- The AGENTS.md Field Guide, 2026 edition · 🟡 S
- AGENTS.md vs CLAUDE.md vs Cursor Rules vs Copilot (2026) · 🟡 S
- Agent instruction files and cross-tool portability — Codex KB · 🟡 S
- AI Harness Engineering Compatibility Matrix — June 2026 · 🟡 S
- Which AI tools actually support MCP well — MCP Bundles, May 2026 · 🟡 S
- Best MCP clients in 2026 — Nimbalyst · 🟡 S
- CLI coding agents comparison — hidekazu-konishi.com · 🟡 S
- OpenCode plugin development: custom tools & event hooks · 🟡 S